

## Aide-Mémoire (VII) Manipulation de Données avec Pandas

### 1. Structures de données Pandas

Pandas dispose de deux types principaux de structures de données, les DataFrames et les series. Un DataFrame est une structure bidimensionnelle, similaire à une table de base de données ou une feuille Excel. Chaque colonne dans un DataFrame est une serie.

### 2. Chargement des données

Pandas permet de charger des données dans des DataFrames à partir de plusieurs types de structures de données et de fichiers. Ci-dessous un tableau récapitulatif des différentes façons de création d'un DataFrame :

| Méthode                    | Exemple                                                            |
|----------------------------|--------------------------------------------------------------------|
| Depuis un dictionnaire     | <code>df = pd.DataFrame({'col1': [1,2], 'col2': [3,4]})</code>     |
| Depuis une liste de listes | <code>df = pd.DataFrame([[1,2], [3,4]], columns=['A', 'B'])</code> |
| Fichier CSV                | <code>df = pd.read_csv('fichier.csv')</code>                       |
| Fichier Excel              | <code>df = pd.read_excel('fichier.xlsx')</code>                    |
| Fichier JSON               | <code>df = pd.read_json('fichier.json')</code>                     |
| Fichier texte              | <code>df = pd.read_table('fichier.txt')</code>                     |
| Fichier HTML               | <code>df_list = pd.read_html('https://exemple.com')</code>         |
| Base de données SQL        | <code>df = pd.read_sql('SELECT * FROM table', con)</code>          |
| Fichier pickle             | <code>df = pd.read_pickle('fichier.pkl')</code>                    |

#### Exemple 1 :

Quand un dictionnaire est utilisé pour le chargement de données, les clés du dictionnaire sont considérées comme les entêtes des différentes colonnes, et la valeur de chaque clé est une liste qui contiendra les valeurs de la colonne correspondante.

```
>>import pandas as pd
>>df = pd.DataFrame({"nom": ["Ahmed", "Salim", "Mounir"], "age": [23, 22, 25]})
```

l'affichage du dataframe se fait sous format matricielle :

```
>>df
   nom  age
0  Ahmed  23
1  Salim  22
2  Mounir  25
```

#### Exemple 2 :

Quand un fichier csv est utilisé pour le chargement de données, chaque colonne est considérée comme une série à injecter dans le dataframe.

```
>>import pandas as pd
>>df = pd.DataFrame(personnes.csv)
>>df
   nom  age
0  Ahmed  23
1  Salim  22
2  Mounir  25
```

### 3. Exploration des données

| Opération                                                | Exemple                                                     |
|----------------------------------------------------------|-------------------------------------------------------------|
| Aperçu des 5 premières lignes                            | <code>df.head()</code>                                      |
| Info sur le DataFrame                                    | <code>df.info()</code>                                      |
| Statistiques descriptives                                | <code>df.describe()</code>                                  |
| Dimensions                                               | <code>df.shape</code>                                       |
| Liste des colonnes                                       | <code>df.columns</code>                                     |
| Types des colonnes                                       | <code>df.dtypes</code>                                      |
| Valeurs uniques d'une colonne                            | <code>df['col'].unique()</code>                             |
| Comptage de valeurs                                      | <code>df['col'].value_counts()</code>                       |
| Récupérer une colonne                                    | <code>df['age']</code>                                      |
| Récupérer plusieurs colonnes                             | <code>df[['age', 'nom']]</code>                             |
| Filtrer le dataframe avec une condition                  | <code>df[df['age'] &gt; 30]</code>                          |
| Filtrer le dataframe avec des conditions multiples (AND) | <code>df[(df['age']&gt;20) &amp; (df['age']&lt;=23)]</code> |
| Conditions multiples (OR)                                | <code>df[(df['age']&lt;20)   (df['age']&gt;23)]</code>      |

La syntaxe `df[condition][ "colonne" ]` crée une copie de la zone correspondante du dataframe et n'est pas donc compatible pour modifier le DataFrame d'origine. A la place, on peut utiliser `df.loc`.

| Opération                                               | Exemple                                                |
|---------------------------------------------------------|--------------------------------------------------------|
| Récupérer une ligne en fonction de son index            | <code>df.loc[0]</code>                                 |
| Récupérer une plage de lignes                           | <code>df.loc[0:2]</code>                               |
| Récupérer toutes les lignes pour quelques colonnes      | <code>df.loc[:, ['age', 'nom']]</code>                 |
| Récupérer toutes les lignes qui vérifient une condition | <code>df.loc[df["age"] &gt; 20, ['age', 'nom']]</code> |
| Modifier une colonne selon un condition                 | <code>df.loc[df["age"] &gt; 20, 'age']+=1</code>       |
| Récupérer une cellule (Ligne + colonne)                 | <code>df.loc[0, 'age']</code>                          |

#### Exemple 1 (accès par position) :

```
>>df = pd.DataFrame({"nom": ["Ahmed", "Salim", "Mounir"], "age": [23, 22, 25]})
>>df.loc(0) #indexation par défaut (0,1,2,...)
nom age
0 Ahmed 23
1 Salim 22
2 Mounir 25

>>df.loc[0:2, 'age']+=1 ou bien df.iloc[0:3]['age']+=1
>>df
nom age
A Ahmed 24
B Salim 23
C Mounir 26
```

#### Exemple 2 (accès par index) :

```
>>df = pd.DataFrame({"nom": ["Ahmed", "Salim", "Mounir"], "age": [23, 22, 25]},
index=['A', 'B', 'C'])
>>df
nom age
A Ahmed 23
B Salim 22
C Mounir 25

>>df.loc['A':'C', 'age']+=1
>>df
nom age
A Ahmed 24
B Salim 23
C Mounir 26
```

#### 4. Manipulations de base

| Opération                        | Exemple                                        |
|----------------------------------|------------------------------------------------|
| Ajouter une colonne              | <code>df['double'] = df['age'] * 2</code>      |
| Supprimer une colonne            | <code>df.drop('age', axis=1)</code>            |
| Renommer colonnes                | <code>df.rename(columns={'nom': 'Nom'})</code> |
| Supprimer lignes avec NaN        | <code>df.dropna()</code>                       |
| Remplacer les valeurs manquantes | <code>df.fillna(0)</code>                      |
| Supprimer doublons               | <code>df.drop_duplicates()</code>              |

#### 5. Manipulations avancées

Pandas permet d'interroger un DataFrame de manière similaire aux requêtes SQL. Ci-dessous, nous présentons un tableau de correspondance entre les commandes SQL et leurs équivalents en Pandas

| Opération               | SQL                                                                 | Pandas                                                                        |
|-------------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Projection              | <code>SELECT col1, col2 FROM table</code>                           | <code>df[['col1', 'col2']]</code>                                             |
| Sélection               | <code>SELECT * FROM table WHERE col1&gt;5 and AND col2&lt;10</code> | <code>df[(df['col1'] &gt; 5) &amp; (df['col2'] &lt; 10)]</code>               |
| Tri                     | <code>ORDER BY col ASC/DESC</code>                                  | <code>df.sort_values('col', ascending=True/False)</code>                      |
| Suppression de doublons | <code>SELECT DISTINCT col FROM table</code>                         | <code>df['col'].drop_duplicates()</code> ou <code>df.drop_duplicates()</code> |
| Renommer des colonnes   | <code>AS</code>                                                     | <code>df.rename(columns={'old': 'new'})</code>                                |
| Agrégation              | <code>SELECT AVG(col) FROM table</code>                             | <code>df['col'].mean()</code>                                                 |
| Groupeement             | <code>Select SUM(col3) GROUP BY col1, col2</code>                   | <code>df.groupby(['col1', 'col2'])['col3'].sum()</code>                       |
| Jointure interne        | <code>INNER JOIN</code>                                             | <code>pd.merge(df1, df2, on='col')</code>                                     |
| Jointure externe        | <code>FULL OUTER JOIN</code>                                        | <code>pd.merge(df1, df2, on='col', how='outer')</code>                        |
| Jointure gauche         | <code>LEFT JOIN</code>                                              | <code>pd.merge(df1, df2, on='col', how='left')</code>                         |
| Jointure droite         | <code>RIGHT JOIN</code>                                             | <code>pd.merge(df1, df2, on='col', how='right')</code>                        |
| Union (sans doublons)   | <code>SELECT * FROM table1 UNION SELECT * FROM table2</code>        | <code>pd.concat([df1, df2]).drop_duplicates()</code>                          |
| Union (avec doublons)   | <code>UNION ALL</code>                                              | <code>pd.concat([df1, df2])</code>                                            |
| Intersection            | <code>SELECT * FROM table1 INTERSECT SELECT * FROM table2</code>    | <code>pd.merge(df1, df2)</code>                                               |
| Différence              | <code>SELECT * FROM table1 EXCEPT SELECT * FROM table2</code>       | <code>df1[~df1.isin(df2)].dropna()</code>                                     |